

Chapter 6.

Recurrent Neural Networks - RNNs

PhD. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn



Sequence data

"**Sequence data**" is a very broad term that refers to any data where the order of elements matters. The specific meaning and interpretation of sequence data depend heavily on the context in which it is being used.

Common types and examples of sequence data:

- Biological Sequence Data: DNA sequences, RNA sequences, Protein sequences.
- Time Series Data: Stock prices, weather data, sensor readings, audio and video signals
- Text Data: Documents, Code, Social media posts
- Other Types of Sequence Data: User activity log, network traffic data, musical scores

•

Key Characteristics of Sequence Data:

- Order matters: The position of an element in the sequence is important and affects its meaning.
- Dependencies: Elements in a sequence can be dependent on preceding or succeeding elements.
- Variable length: Sequences can have different lengths.

Examples of sequence data

Speech recognition		→	 "The quick brown fox jumped over the lazy dog."
Music generation		→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCTGTGAGGAAGTAG	→	AGCCCTGTGAGGAAGTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger . Andrew Ng

Recurrent Neural Networks

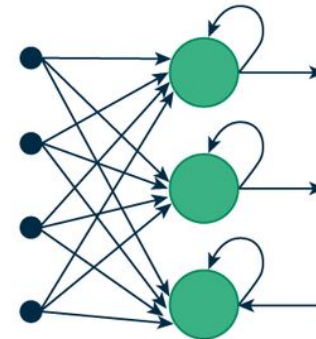
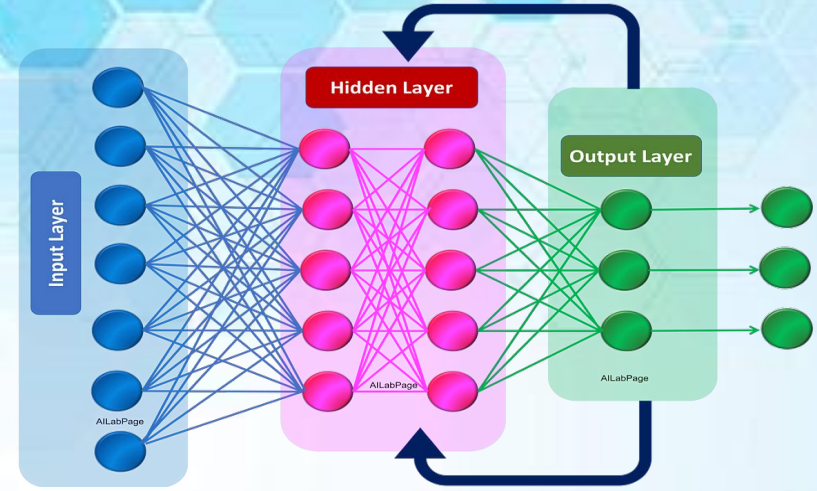
Recurrent Neural Networks (RNNs) are a type of neural network designed specifically to process sequential data. Unlike traditional feedforward neural networks that process inputs independently, RNNs have a mechanism that allows them to maintain a "memory" of previous inputs in the sequence. This makes them well-suited for tasks where the order and temporal relationships between data points are important.

Key Features of RNNs:

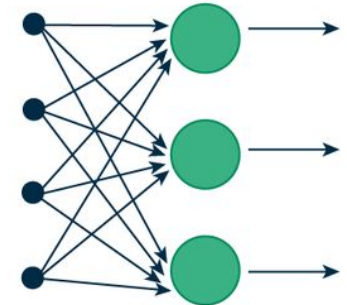
Recurrent Connections: The core idea of an RNN is the presence of recurrent connections. This means that the output of a layer at one time step can be fed back as input to the same layer at the next time step. This creates a loop that allows information to persist across the sequence.

Hidden State: RNNs maintain a hidden state (also sometimes called a memory state) that acts as a "memory" of the sequence seen so far. This hidden state is updated at each time step based on the current input and the previous hidden state.

Processing Sequences: RNNs process sequences element by element, one at a time. At each time step, the network takes the current input and the previous hidden state to compute the current hidden state and an output (if required).



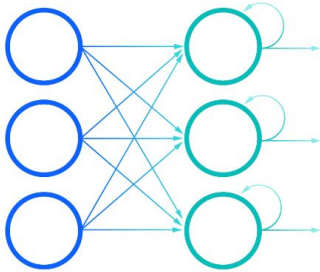
(a) Recurrent Neural Network



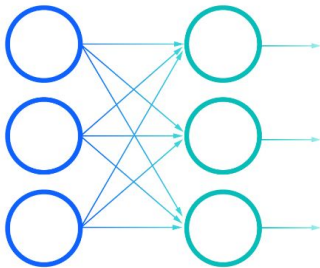
(b) Feed-Forward Neural Network

How RNNs work

Recurrent Neural Networks



Feedforward Neural Networks



Rolled RNN

Output layer



Hidden layers

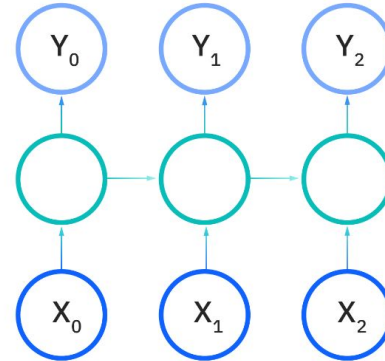


Input layer



=

Unrolled RNN



Time

How RNNs Handle Sequence Data

1. **Sequential Processing:** RNNs process the input sequence one element at a time. For each element in the sequence, the RNN takes two inputs:

- The current input element.
- The hidden state from the previous time step.

2. **Hidden State Update:** The RNN uses these two inputs to compute a new hidden state. This computation involves:

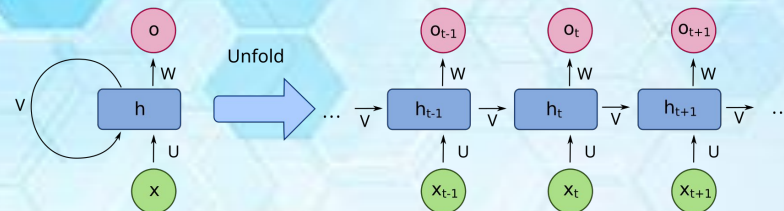
- Weight matrices that are learned during training. These weights determine how the current input and the previous hidden state influence the new hidden state.
- An activation function (like tanh or ReLU) that introduces non-linearity.

The formula for a basic RNN hidden state update at time step t (h_t) looks like this:

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h)$$

where:

- x_t is the input at time step t .
- h_{t-1} is the hidden state from the previous time step (h_0 is typically initialized to zeros).
- W_{ih} is the weight matrix connecting the input to the hidden layer.
- W_{hh} is the weight matrix connecting the previous hidden state to the current hidden layer (this is what enables the memory).
- b_h is the bias vector for the hidden layer.
- $\tanh$ is the activation function.



3. **Information Propagation:** The hidden state at each time step carries information about the entire sequence seen so far. The influence of earlier elements gradually gets incorporated into the hidden state as the network processes the sequence.

4. **Output Generation (Optional):** At each time step (or at the end of the sequence), the RNN can also produce an output based on the current hidden state. This output can be used for various tasks, such as predicting the next element in the sequence, classifying the entire sequence, or generating a corresponding output sequence. The output is typically computed using another weight matrix and an activation function:

$$o_t = W_{ho}h_t + b_o$$

where:

- o_t is the output at time step t .
- W_{ho} is the weight matrix connecting the hidden layer to the output layer.
- b_o is the bias vector for the output layer.

Common Architectures of RNNs

Common Architectures of RNNs:

-**Simple RNN (Vanilla RNN):** The basic form of an RNN with a single recurrent layer.

-**Long Short-Term Memory (LSTM) Networks:** A more sophisticated type of RNN that addresses the vanishing gradient problem, allowing it to learn long-range dependencies in sequences. LSTMs have special "gates" (input gate, forget gate, output gate) that control the flow of information into and out of the memory cell.

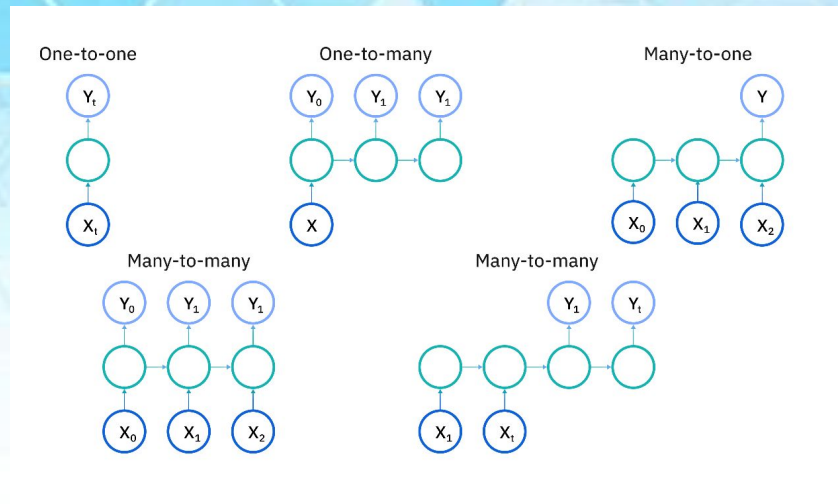
-**Gated Recurrent Unit (GRU) Networks:** Another type of RNN with gating mechanisms, but with a simpler architecture than LSTMs (fewer gates). GRUs often perform comparably to LSTMs and are computationally more efficient.

-**Bidirectional RNNs:** These RNNs process the input sequence in both forward and backward directions, allowing them to capture information from both past and future contexts.

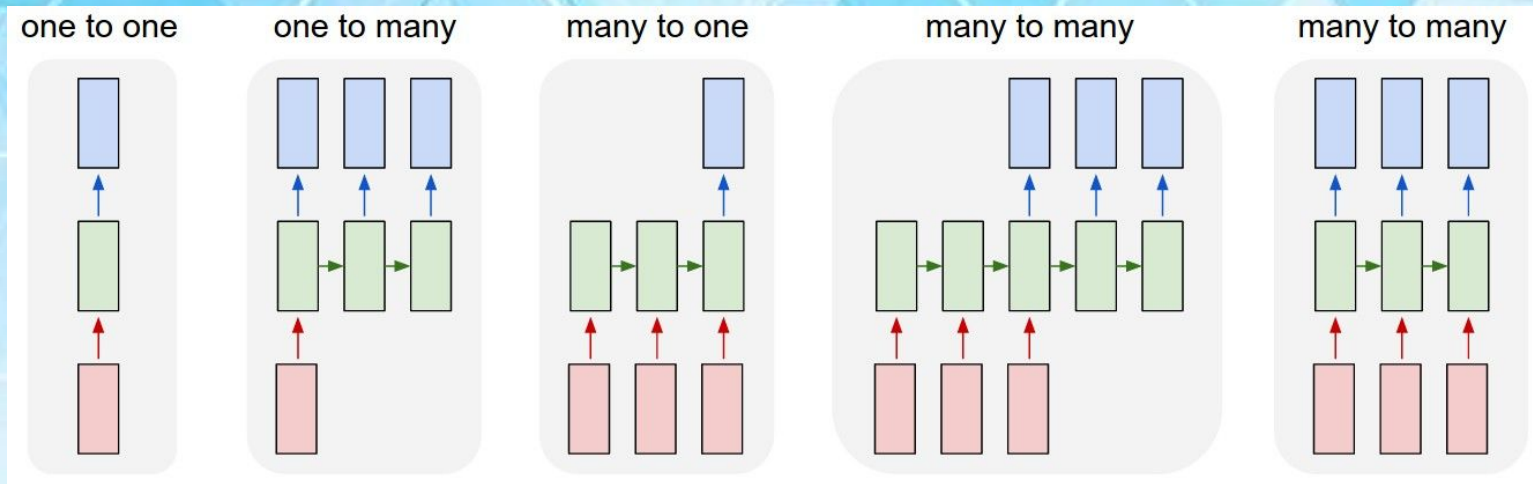
-Encoder-Decoder RNNs:

Limitations of Simple RNNs:

- **Vanishing and Exploding Gradients:** Simple RNNs can struggle to learn long-range dependencies due to the vanishing or exploding gradient problem during training. This is why more advanced architectures like LSTMs and GRUs were developed.
- **Difficulty in Capturing Long-Term Dependencies:** While recurrent connections allow for memory, simple RNNs often have difficulty remembering information over very long sequences.



Common Architectures of RNNs



Each rectangle is a vector and arrows represent functions (e.g. matrix multiply). Input vectors are in red, output vectors are in blue and green vectors hold the RNN's state .

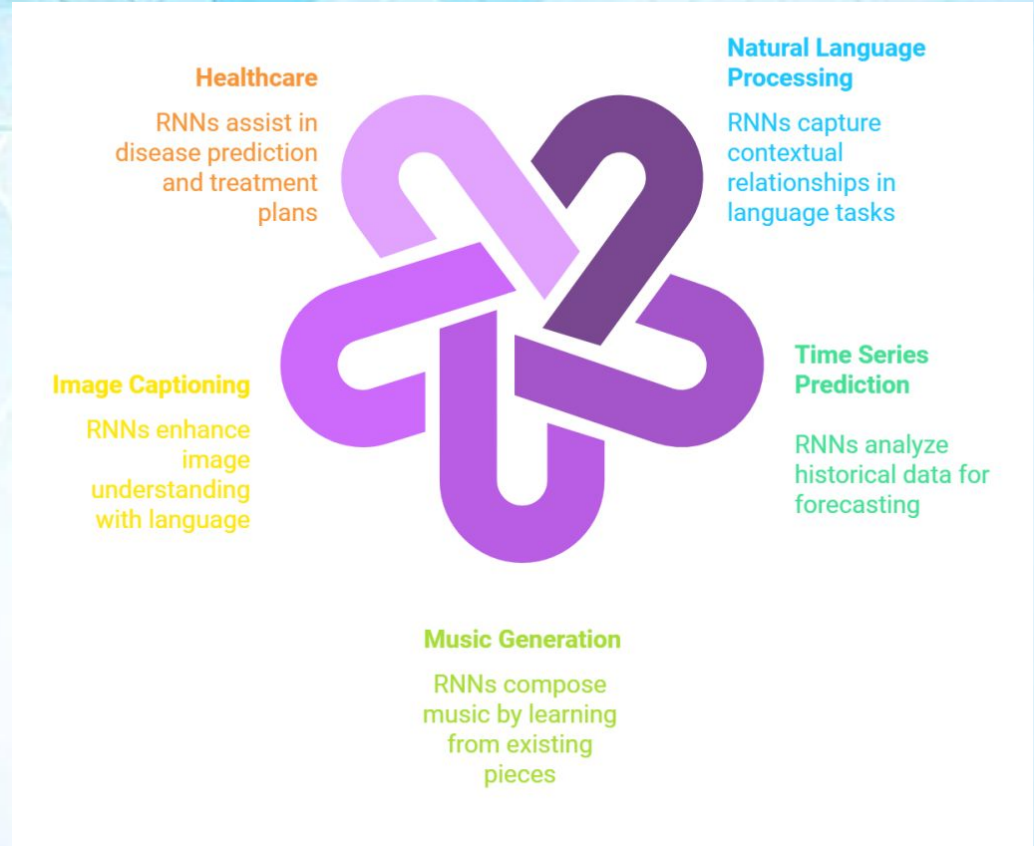
- (1) Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).
- (2) Sequence output (e.g. image captioning takes an image and outputs a sentence of words).
- (3) Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).
- (4) Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).
- (5) Synced sequence input and output (e.g. video classification where we wish to label each frame of the video). Notice that in every case are no pre-specified constraints on the lengths sequences because the recurrent transformation (green) is fixed and can be applied as many times as we like.

Applications of RNNs

RNNs excel at processing sequences, making them vital for:

- **NLP:** Language modeling, translation, sentiment analysis, text generation.
- **Speech:** Recognition, synthesis.
- **Time Series:** Prediction (stocks, weather), anomaly detection.
- **Video:** Captioning, action recognition.
- **Other:** Music/DNA sequence generation, robot control.

LSTMs and GRUs, advanced RNNs, often power these applications.



Recurrent Neural Network with Pytorch

SimpleRNN(nn.Module): We define our RNN as a class inheriting from `nn.Module`, PyTorch's base class for neural network modules.

`__init__(self, input_size, hidden_size, output_size):`

- `input_size`: The number of features in each input vector at a single time step.
- `hidden_size`: The number of hidden units, which determines the capacity of the RNN's memory.
- `output_size`: The number of possible output classes or the size of the output vector.
- `self.i2h = nn.Linear(input_size + hidden_size, hidden_size)`: This is the linear layer that combines the current input with the previous hidden state to produce the next hidden state.
- `self.h2o = nn.Linear(hidden_size, output_size)`: This linear layer maps the hidden state to the output.
- `self.softmax = nn.LogSoftmax(dim=1)`: A softmax activation function (often used for classification) applied to the output. We use `LogSoftmax` which is numerically more stable when used with `nn.NLLLoss`.

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleRNN(nn.Module):
5     def __init__(self, input_size, hidden_size, output_size):
6         super(SimpleRNN, self).__init__()
7         self.hidden_size = hidden_size
8         # Input-to-hidden layer
9         self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
10        # Hidden-to-output layer
11        self.h2o = nn.Linear(hidden_size, output_size)
12        # Output activation
13        self.softmax = nn.LogSoftmax(dim=1)
14
15    def forward(self, input_tensor, hidden):
16        # Combine input and previous hidden state
17        combined = torch.cat((input_tensor, hidden), 1)
18        # Calculate the new hidden state
19        hidden = torch.tanh(self.i2h(combined))
20        # Calculate the output
21        output = self.softmax(self.h2o(hidden))
22        return output, hidden
23
24    def init_hidden(self):
25        # Initialize the hidden state with zeros
26        return torch.zeros(1, self.hidden_size)
```

Recurrent Neural Network with Pytorch

forward(self, input_tensor, hidden): This method defines how the input data flows through the network.

- `combined = torch.cat((input_tensor, hidden), 1)`: The current input and the previous hidden state are concatenated along the feature dimension (dimension 1).
- `hidden = torch.tanh(self.i2h(combined))`: The combined vector is passed through the input-to-hidden linear layer, and then a `tanh` (hyperbolic tangent) activation function is applied to produce the new hidden state. The `tanh` function squashes the values between -1 and 1, helping to regulate the gradients.
- `output = self.softmax(self.h2o(hidden))`: The new hidden state is passed through the hidden-to-output linear layer, and then the `LogSoftmax` function is applied to get the output probabilities (in log space).
- The function returns both the `output` and the updated `hidden` state. The hidden state needs to be passed to the next time step.

init_hidden(self): This method initializes the hidden state to a tensor of zeros.¹ The hidden state is typically initialized before processing each sequence. The shape `(1, self.hidden_size)` represents a single hidden state for one sequence. When processing a batch, we'll repeat this initial hidden state for each sequence in the batch.

```
1 # Define hyperparameters
2 input_size = 10 # Size of each input feature
3 hidden_size = 20 # Number of hidden units
4 output_size = 5 # Number of output classes
5 sequence_length = 5 # Length of the input sequence
6 batch_size = 2 # Number of sequences in a batch
7
8 # Create an instance of the SimpleRNN
9 rnn = SimpleRNN(input_size, hidden_size, output_size)
10
11 # Create a sample input sequence
12 input_seq = torch.randn(batch_size, sequence_length, input_size)
13
14 # Initialize the hidden state
15 hidden = rnn.init_hidden().repeat(batch_size, 1) # For the whole batch
16
17 # Process the sequence
18 for i in range(sequence_length):
19     output, hidden = rnn(input_seq[:, i, :], hidden)
20
21 # The final output and hidden state after processing the entire sequence
22 print("Final Output Shape:", output.shape)
23 print("Final Hidden State Shape:", hidden.shape)
```

✓ 0.0s

Final Output Shape: torch.Size([2, 5])
Final Hidden State Shape: torch.Size([2, 20])

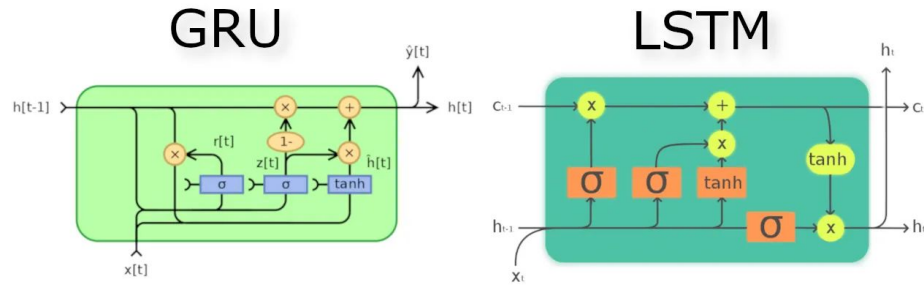
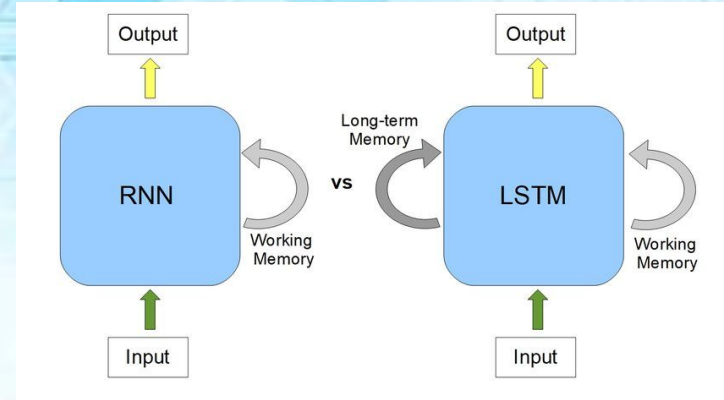
Long-Short Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a type of Recurrent Neural Network (RNN) that excel at learning long-term dependencies in sequential data. Traditional RNNs often struggle with the "vanishing gradient" problem, making it difficult for them to learn from information that lies far back in the sequence. LSTMs were specifically designed to address this limitation.

Variations of LSTMs:

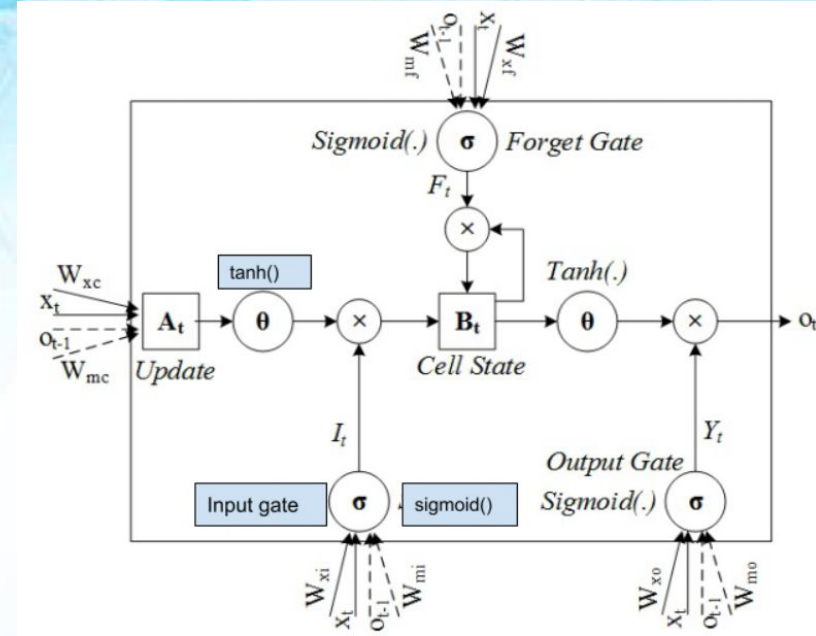
There are several variations of the standard LSTM architecture, such as:

- **Gated Recurrent Units (GRUs):** A simplified version of LSTMs with fewer gates.
- **Bidirectional LSTMs (BiLSTMs):** Process the input sequence in both forward and backward directions to capture context from both past and future.
- **Convolutional LSTMs (ConvLSTMs):** Combine convolutional layers with LSTM layers to process spatiotemporal data.
- **LSTM with Attention Mechanisms:** Allow the model to focus on the most relevant parts of the input sequence when making predictions.



Architecture of an LSTM Cell

- **Input (X_t) and Previous Hidden State (O_{t-1}):** These are the inputs to the GRU cell at the current time step.
- **Update Gate:** Controlled by a sigmoid function (σ), it determines how much of the previous hidden state to retain and how much of the new candidate hidden state to incorporate. It uses weights W_{xc} and W_{mc} .
- **Candidate Hidden State ($h \sim t$):** Calculated using a $\tanh$ function, it proposes a new hidden state based on the current input and the previous hidden state (modulated by the update gate). It involves weights W_{xc} and W_{mc} .
- **Reset Gate:** (Implicitly combined with the update gate in this diagram simplification compared to some GRU variations). It typically helps decide how much of the past hidden state to consider when calculating the candidate hidden state.
- **New Hidden State (O_t):** A weighted combination of the previous hidden state and the candidate hidden state, determined by the update gate.
- **Output Gate:** (Labelled as "Output Gate Sigmoid(.)" but its output Y_t is directly used to compute O_t in this simplified GRU). Controlled by another sigmoid function with weights W_{xo} and W_{mo} , it determines which parts of the cell state are outputted as the new hidden state.



LSTM with Pytorch

SimpleLSTM(nn.Module): Defines our LSTM model as a PyTorch module.

__init__(self, input_size, hidden_size, num_layers, output_size):

- **input_size**: The number of features per input time step.
- **hidden_size**: The number of hidden units in each LSTM layer.
- **num_layers**: The number of stacked LSTM layers. More layers can capture more complex patterns.
- **output_size**: The size of the final output (e.g., number of classes).
- **self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)**: This is PyTorch's built-in LSTM layer.
 - **batch_first=True** ensures that the input and output tensors have the batch size as the first dimension (batch, sequence, features), which is often more convenient.
- **self.fc = nn.Linear(hidden_size, output_size)**: A fully connected layer to map the final hidden state to the desired output size.

init_hidden(self, batch_size):

- This method initializes the hidden state and cell state tensors with zeros.
- The shape (**num_layers**, **batch_size**, **hidden_size**) is crucial for the LSTM layer. The first dimension represents the number of layers, the second the batch size (number of independent sequences being processed in parallel), and the third the number of hidden units.

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleLSTM(nn.Module):
5     def __init__(self, input_size, hidden_size, num_layers, output_size):
6         super(SimpleLSTM, self).__init__()
7         self.hidden_size = hidden_size
8         self.num_layers = num_layers
9         # PyTorch's built-in LSTM layer
10        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
11        # Fully connected layer to map hidden state to output
12        self.fc = nn.Linear(hidden_size, output_size)
13
14    def forward(self, input_seq, hidden):
15        # Input shape: (batch_size, seq_len, input_size)
16        # Hidden shape: (num_layers, batch_size, hidden_size) - initialized outside
17        # Cell state shape: (num_layers, batch_size, hidden_size) - initialized outside
18
19        # out shape: (batch_size, seq_len, hidden_size)
20        # hidden shape: (num_layers, batch_size, hidden_size)
21        # cell state shape: (num_layers, batch_size, hidden_size)
22        out, hidden = self.lstm(input_seq, hidden)
23
24        # We often want the output of the last time step for sequence classification
25        # out[:, -1, :] has shape (batch_size, hidden_size)
26        output = self.fc(out[:, -1, :])
27        return output, hidden
28
29    def init_hidden(self, batch_size):
30        # Initialize hidden and cell state with zeros
31        # Shape: (num_layers, batch_size, hidden_size)
32        h0 = torch.zeros(self.num_layers, batch_size, self.hidden_size)
33        c0 = torch.zeros(self.num_layers, batch_size, self.hidden_size)
34        return (h0, c0)
```

LSTM with Pytorch

`forward(self, input_seq, hidden):`

- `input_seq`: The input sequence tensor with shape `(batch_size, seq_len, input_size)`.
- `hidden`: A tuple containing the initial hidden state `h_0` and cell state `c_0`, both with shape `(num_layers, batch_size, hidden_size)`. These are initialized outside the forward pass.
- `out, hidden = self.lstm(input_seq, hidden)`: The input sequence and the initial hidden/cell states are passed through the LSTM layer.
 - `out`: The output features from the last layer of the LSTM for each time step. Its shape is `(batch_size, seq_len, hidden_size)`.
 - `hidden`: A tuple containing the final hidden state and cell state after processing the entire sequence. Their shapes are both `(num_layers, batch_size, hidden_size)`.
- `output = self.fc(out[:, -1, :])`: For many sequence classification tasks, we are interested in the output at the last time step. `out[:, -1, :]` selects the output of the last time step for all batches. This is then passed through the fully connected layer to get the final prediction.
- The function returns the final `output` and the final `hidden` state (a tuple of the last hidden state and cell state). You might need the final hidden state for further processing of sequences.

```
1 # --- Example Usage ---
2 # Define hyperparameters
3 input_size = 10 # Size of each input feature
4 hidden_size = 20 # Number of hidden units
5 num_layers = 2 # Number of LSTM layers stacked
6 output_size = 5 # Number of output classes
7 sequence_length = 5 # Length of the input sequence
8 batch_size = 3 # Number of sequences in a batch
9
10 # Create an instance of the SimpleLSTM
11 lstm_model = SimpleLSTM(input_size, hidden_size, num_layers, output_size)
12
13 # Create a sample input sequence
14 input_seq = torch.randn(batch_size, sequence_length, input_size)
15
16 # Initialize hidden and cell states
17 hidden = lstm_model.init_hidden(batch_size)
18
19 # Forward pass
20 output, hidden = lstm_model(input_seq, hidden)
21
22 # Print output shapes
23 print("Output Shape:", output.shape) # (batch_size, output_size)
24 print("Hidden State Shape:", hidden[0].shape) # (num_layers, batch_size, hidden_size)
25 print("Cell State Shape:", hidden[1].shape) # (num_layers, batch_size, hidden_size)
✓ 0.0s
```

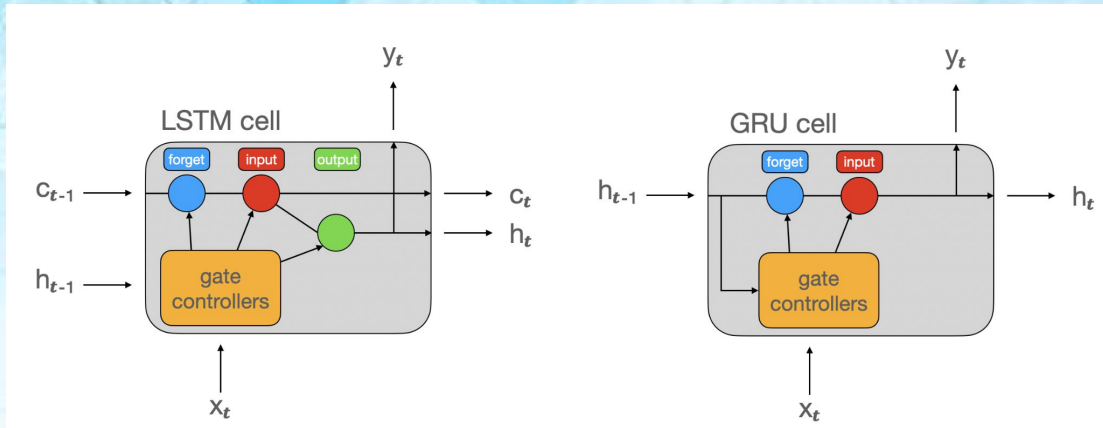
Output Shape: torch.Size([3, 5])
Hidden State Shape: torch.Size([2, 3, 20])
Cell State Shape: torch.Size([2, 3, 20])

Gated recurrent units (GRUs)

Gated Recurrent Units (GRUs) are a type of RNN that were introduced as a simplification of the Long Short-Term Memory (LSTM) architecture. Like LSTMs, GRUs are designed to learn long-term dependencies in sequential data and address the vanishing gradient problem encountered in traditional RNNs.

Key Differences from LSTMs:

- **Single Hidden State:** GRUs have only one hidden state that is passed between time steps, whereas LSTMs have both a hidden state and a cell state.
- **Fewer Gates:** GRUs have two gates (update and reset) compared to the three gates (forget, input, and output) in standard LSTMs.
- **Simpler Structure:** The reduced number of parameters and simpler gating mechanism can make GRUs computationally more efficient and sometimes easier to train than LSTMs, especially on smaller datasets.



Advantages of GRUs:

- **Handles Vanishing Gradients:** Similar to LSTMs, the gating mechanisms allow GRUs to learn long-term dependencies.
- **Computationally Efficient:** Fewer parameters and a simpler structure can lead to faster training and inference times compared to LSTMs.
- **Good Performance:** GRUs often achieve performance comparable to LSTMs on many sequence modeling tasks.

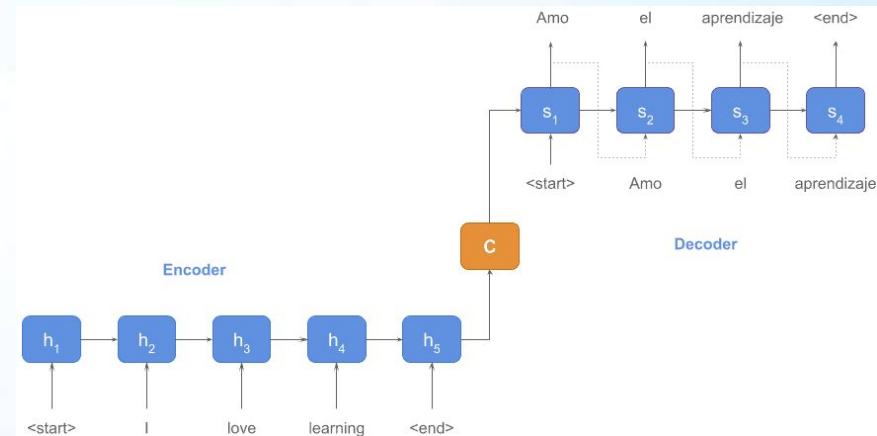
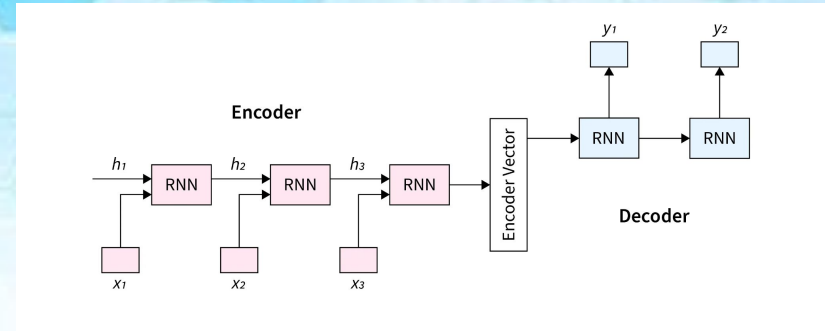
Encoder-decoder RNN

Encoder-decoder RNNs are a powerful architecture for sequence-to-sequence tasks, where the input and output are both sequences of potentially different lengths. Think of tasks like machine translation (English sentence to French sentence) or text summarization (long document to a shorter summary).

The encoder-decoder architecture consists of **two main parts**:

Encoder: An RNN (could be a simple RNN, LSTM, or GRU) that processes the input sequence step by step. As it reads each element of the input sequence, the encoder updates its internal hidden state. The goal of the encoder is to compress the entire input sequence into a single, fixed-length vector. This vector, often called the context vector or thought vector, is meant to be a rich representation of the meaning of the entire input sequence.

Decoder: Another RNN that takes the context vector produced by the encoder as its initial hidden state. The decoder then generates the output sequence one element at a time. At each step, the decoder uses its current hidden state and the previously generated output (or a special start-of-sequence token initially) to predict the next element in the output sequence. This process continues until the decoder generates a special end-of-sequence token, indicating the completion of the output.



Practice

<https://github.com/karpathy/char-rnn>

<https://github.com/deependuiha/Word-Embedding-PyTorch>

Sequence representation for RNNs involves converting your sequential data into a numerical format that the model can understand.

Key techniques include:

- **Numerical Encoding:**
 - **Integer Encoding:** Assigning unique integers to distinct items (e.g., words, categories).
 - **One-Hot Encoding:** Creating binary vectors for each item, indicating its presence.
 - **Embeddings:** Learning dense, low-dimensional vector representations capturing semantic relationships (especially for high-cardinality categorical data like words).
- **Handling Variable Lengths:**
 - **Padding:** Adding special tokens to shorter sequences to match the length of the longest.
 - **Truncation:** Shortening longer sequences to a fixed maximum length.